

Aluno: Rafael Holder de Oliveira

GitHub: <https://github.com/rafaelholder/CESAR-2026.2-ia-ciberseguranca>.

Evidências de funcionamento do Notebook

- Drive

```
Passo 3 — Organizar no Drive

Vamos colocar o modelo em models/ e as imagens de teste em data/test/<nome-da-classe>/, usando exata-
mente os nomes de classe do Teachable Machine.

1] 17s
from google.colab import drive
drive.mount('/content/drive')

from pathlib import Path
PROJETO = Path('/content/drive/MyDrive/ia-ciberseguranca')
(PROJETO / 'models').mkdir(parents=True, exist_ok=True)
(PROJETO / 'data' / 'test').mkdir(parents=True, exist_ok=True)

print('Coloque converted_keras.zip em:', PROJETO / 'models')
print('Coloque as imagens de teste em: ', PROJETO / 'data' / 'test' / '<classe>')

Mounted at /content/drive
Coloque converted_keras.zip em: /content/drive/MyDrive/ia-ciberseguranca/models
Coloque as imagens de teste em: /content/drive/MyDrive/ia-ciberseguranca/data/test/<classe>
```

- Inserindo dados prontos para o modelo (Tirados do TensorFlow)

```
[2] 11s
from pathlib import Path
import random, shutil, zipfile, urllib.request

PROJETO = Path('/content/drive/MyDrive/ia-ciberseguranca')
random.seed(42)

URLS = {
    'rps.zip': 'https://storage.googleapis.com/download.tensorflow.org/data/rps.zip',
    'rps-test-set.zip': 'https://storage.googleapis.com/download.tensorflow.org/data/rps-test-set.zip',
}
for nome, url in URLS.items():
    urllib.request.urlretrieve(url, nome)
    zipfile.ZipFile(nome).extractall('.')

CLASSES = ['rock', 'paper', 'scissors']
N_TREINO, N_TESTE = 70, 15

for c in CLASSES:
    treino = sorted(Path('rps', c).glob('*.png'))
    teste = sorted(Path('rps-test-set', c).glob('*.png'))
    dst_tr = PROJETO / 'data' / 'tm_treino' / c
    dst_te = PROJETO / 'data' / 'test' / c
    dst_tr.mkdir(parents=True, exist_ok=True)
    dst_te.mkdir(parents=True, exist_ok=True)
    for f in random.sample(treino, N_TREINO):
        shutil.copy(f, dst_tr / f.name)
    for f in random.sample(teste, N_TESTE):
        shutil.copy(f, dst_te / f.name)
    print(f'{c:9s} -> {N_TREINO} treino | {N_TESTE} teste')

... rock      -> 70 treino | 15 teste
    paper     -> 70 treino | 15 teste
    scissors  -> 70 treino | 15 teste
```

- Modelo existe no drive

```
[3]
✓ 0s # Descompacta o modelo exportado do Teachable Machine
import zipfile

ZIP = PROJETO / 'models' / 'converted_keras.zip'
DEST = PROJETO / 'models'
with zipfile.ZipFile(ZIP) as z:
    z.extractall(DEST)

H5 = DEST / 'keras_model.h5'
LABELS = DEST / 'labels.txt'
print('Modelo :', H5.exists(), H5)
print('Labels :', LABELS.exists(), LABELS)

... Modelo : True /content/drive/MyDrive/ia-ciberseguranca/models/keras_model.h5
Labels : True /content/drive/MyDrive/ia-ciberseguranca/models/labels.txt
```

- Carregando modelo

```
[4]
16s # Carrega modelos Keras 2 (Teachable Machine) sob TF novo
os.environ["TF_USE_LEGACY_KERAS"] = "1"
!pip install -q tf_keras

import tensorflow as tf
from tensorflow.keras.layers import DepthwiseConv2D
print('TensorFlow', tf.__version__)

# Patch: versões antigas salvam 'groups' no DepthwiseConv2D; o Keras novo não entende
class PatchedDepthwiseConv2D(DepthwiseConv2D):
    def __init__(self, *args, **kwargs):
        kwargs.pop('groups', None)
        super().__init__(*args, **kwargs)

model = tf.keras.models.load_model(
    H5, compile=False,
    custom_objects={'DepthwiseConv2D': PatchedDepthwiseConv2D})

class_names = [l.strip().split(' ', 1)[-1]
                for l in open(LABELS, encoding='utf-8') if l.strip()]
print('Classes:', class_names)
model.summary()
```

```
... TensorFlow 2.20.0
Classes: ['Rock', 'Paper', 'Scissors']
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	(None, 1280)	410208
sequential_3 (Sequential)	(None, 3)	128400

```

Total params: 538608 (2.05 MB)
Trainable params: 524528 (2.00 MB)
Non-trainable params: 14080 (55.00 KB)
```

- Teste

```
[6] ✓ 2s # Teste rápido com UMA imagem (aponte para um arquivo do seu data/test)
import matplotlib.pyplot as plt

exemplos = list((PROJETO / 'data' / 'test').rglob('*'))
exemplos = [p for p in exemplos if p.suffix.lower() in {'.jpg', '.jpeg', '.png'}]
assert exemplos, 'Coloque imagens em data/test/<classe>/ antes de rodar.'

alvo = exemplos[2]
classe, conf, _ = prever(alvo)
plt.imshow(Image.open(alvo)); plt.axis('off')
plt.title(f'Previsto: {classe} ({conf:.1%})')
plt.show()
```

Previsto: Rock (100.0%)

A photograph of a human hand in a rock pose, with the index, middle, and ring fingers extended and the thumb and pinky curled in. The hand is positioned against a plain, light-colored background.

- Matriz de Acúrcia

*** Imagens de teste : 45
Acurácia no teste: 95.6%

	precision	recall	f1-score	support
paper	1.00	0.87	0.93	15
rock	0.88	1.00	0.94	15
scissors	1.00	1.00	1.00	15
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

